be essential to keeping connected products operational throughout their lifecycle.

Device-specific update strategies for scalable FOTA

A scalable FOTA system must support different update strategies based on how devices operate in the field. Broadly, connected devices fall into two categories: always-on devices and battery-powered devices that spend most of their time in sleep mode. Since their operating patterns differ, the firmware delivery method must also differ.

Battery-powered devices wake up in response to an event or on a schedule and communicate with the FOTA server to check for available updates. If an update is available, the device downloads and installs it before returning to sleep mode. This approach is well suited to devices that are not always connected.

On the other hand, always-connected devices do not need to wait for an update from the server. These devices can perform the update at an appropriate time, such as when they are idle or not handling production tasks, preventing unnecessary downtime and disruption in critical operations.

In large-scale deployments involving thousands or even millions of devices, updates must be scheduled so that devices are upgraded only when they are available. This allows the fleet to continue operating while firmware is rolled out in a controlled manner.

The update mechanism also depends on the device architecture. Systems running a full operating system often use delta updates, where only the modified portions of the firmware are transmitted. This reduces download size, saves bandwidth, and speeds up the update process.

Resource-constrained devices, such as microcontrollers based on Arm Cortex-M cores, typically receive a complete firmware image. These devices usually contain a permanent bootloader, while the entire application firmware is replaced during the update. Because the firmware size is relatively small, a full image update is simpler than generating and applying delta patches.

Communication protocols are also selected according to their purpose. MQTT is commonly used for command and control, such as notifying devices that a new firmware version is available. The firmware image itself is typically transferred using HTTPS or FTP, which can handle the reliable delivery of larger files.

Reliable firmware storage and recovery mechanisms

One of the most widely used approaches to reliable firmware storage and recovery is the A/B partitioning architecture. In this design, the microcontroller's flash memory is divided into two firmware partitions. The currently running firmware remains in one partition (Slot A), while the new firmware is downloaded into the other (Slot B). Once the download is complete, the device reboots, and the bootloader verifies the new firmware by checking its integrity and confirming that it starts correctly. If the verification succeeds, the new firmware becomes active. If the firmware fails to boot or does not pass the checks, the bootloader automatically restores the previous version. This rollback mechanism prevents devices from becoming unusable after a failed update.

To simplify fleet management, devices should also report failed updates to the cloud whenever a rollback occurs. This gives developers and operators visibility into update failures, allowing them to identify affected devices and investigate the root cause.

The bootloader plays a central role in update reliability. It confirms that the firmware in flash memory corresponds to the image provided by the server before executing the firmware. It is usually kept in a reserved flash segment such as LDROM (loader ROM) or another dedicated partition that is not modified during regular firmware update processes. This keeps the bootloader available to recover the device if an application update fails.

Flash memory can also include a dedicated partition for storing device configuration data. This allows Wi-Fi credentials, calibration data, user settings, threshold values, and other configuration parameters to be retained during the firmware update, eliminating the need to reconfigure the device after each update.

However, for resource-constrained devices such as

What should you consider to make devices OTA ready?

- ***Plan for rollback from the start.*** Choose a microcontroller with enough memory to store both the current and new firmware. This allows the device to roll back if an update fails.
- ***Allocate sufficient flash memory.*** Limited memory can prevent rollback, increasing the risk of device failures after a faulty update.
- ***Build in secure communication.*** Use secure protocols such as HTTPS/TLS to protect firmware downloads from tampering.
- ***Validate every firmware image.*** Perform CRC (cyclic redundancy check) before installing the firmware to verify that it has not been corrupted during transfer. Do not skip integrity checks.
- ***Avoid fixed server addresses.*** Hardcoding the OTA server or IP address can prevent deployed devices from receiving updates if the server changes or becomes unavailable.
- ***Support dynamic server configuration.*** Allow the OTA server URL or endpoint to be updated remotely so devices can continue receiving firmware updates.